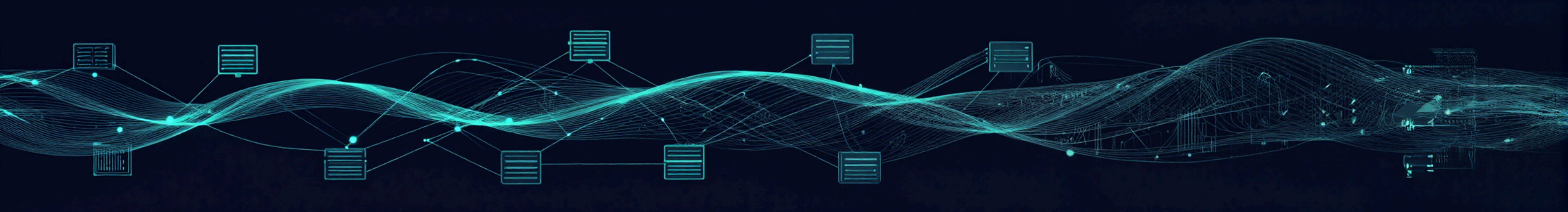




SMARTBANCS TRANSACCIONES SEGURAS + IA ASÍNCRONA

Reto técnico · MVP de Arquitectura Financiera · Java 21 | Spring Boot | PostgreSQL | RabbitMQ | FastAPI | Docker





ARQUITECTURA DE LA SOLUCIÓN

Flujo principal: Cliente → REST API → Transaction Service → PostgreSQL (commit). Tras el commit, se publica un evento en RabbitMQ → AI Risk Service (FastAPI). La IA opera de forma completamente asíncrona, fuera del camino crítico transaccional.

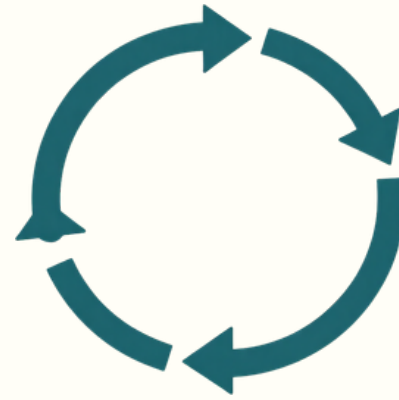
Bancs Legacy se integra de forma desacoplada mediante un proceso ETL independiente, sin afectar la disponibilidad ni la latencia del servicio principal.

Pilares del diseño: separación de responsabilidades, resiliencia ante fallos de IA y trazabilidad end-to-end con transactionId.



BLOQUEO PESIMISTA

SELECT FOR UPDATE garantiza acceso exclusivo a la cuenta durante la transacción. Previene condiciones de carrera en entornos de alta concurrencia. El bloqueo se libera automáticamente al finalizar el commit, asegurando consistencia en el saldo.



ORDEN DETERMINISTA

Los bloqueos se adquieren siempre en orden ascendente por UUID de cuenta. Este orden determinista elimina deadlocks cíclicos entre transacciones concurrentes. El flujo: Idempotency-Key → validación → bloqueo → validación saldo → commit → evento.



IDEMPOTENCIA

Cada solicitud incluye un Idempotency-Key único. Si la clave ya fue procesada, se retorna el resultado original sin ejecutar el movimiento nuevamente. Previene débitos o créditos duplicados ante reintentos del cliente o fallos de red.



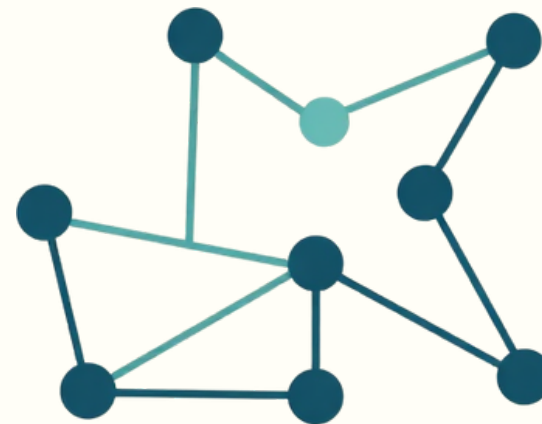
IA, BANCOS Y OBSERVABILIDAD

IA Asíncrona

Evento: transaction.completed publicado tras commit. Consumidor FastAPI procesa el mensaje de forma asíncrona. Modelo Isolation Forest detecta anomalías sin bloquear el camino crítico transaccional. La IA opera en aislamiento total del flujo de pago.

Bancos y Observabilidad

Bancos protegido: batching, rate limiting, retry/backoff y reconciliación. Evolución hacia Transactional Outbox.
Observabilidad: Actuator, Micrometer, Prometheus. Métricas: transactionId, Hikari, RabbitMQ, HTTP. Diagnóstico: latencia, errores, pool DB, pg_stat_activity, pg_locks, backlog RabbitMQ.



Resultado y límites del MVP

✓ Docker Compose: 4 servicios

• Transferencia E2E • RabbitMQ → IA • ETL funcional • Métricas
Prometheus

⚠ 10.000 TPS no validado • IA con datos sintéticos • Sin Transactional
Outbox • Suite de tests limitada

Diseño reproducible, desacoplado y preparado para evolucionar.